S.Y. B. Tech. Academic Year 2026-27 Semester: III

CSE20350/AID20210/CLD 20020: Data Structures Laboratory

WRITE UP

Experiment Number: 06

TITLE: Expression conversion using stack

PROBLEM STATEMENT:

Implement stack as an ADT and apply it for different expression conversions (infix to postfix or infix to prefix (Any one), prefix to postfix or prefix to infix, postfix to infix or postfix to prefix (Any one)).

OBJECTIVE:

1. To study Stack and its operations
2. To study the importance of expression conversions

THEORY

Stack: Special case of ordered list also called as restricted/controlled list where insertion and deletion happen at only one end called as top of stack (homogeneous collection of elements.) Elements are added to and removed from the top of the stack *(the most recently added items are at the top of the stack).* The last element to be added is the first to be removed (**LIFO**: Last In, First Out). Only access to the stack is the top element

Basic Stack operations:

- isEmpty()-Checking stack is empty
- IsFull()-Checking stack is full
- push()-Pushing Element on top of stack
- pop() – Popping top element from stack

Applications of Stack

1. Expression evaluation (Postfix and Prefix)
2. Infix to Postfix and Prefix conversion
3. Function call management (Call Stack)
4. Recursion
5. Parentheses matching
6. Undo/Redo operations in editors
7. Browser back and forward navigation
8. Depth-First Search (DFS) in graphs
9. Syntax parsing in compilers
10. String reversal

Expression conversion is the process of converting an arithmetic expression from one notation to another using a **stack**. The three common notations are:

- **Infix:** Operator is between operands (e.g., A + B)
- **Prefix (Polish):** Operator is before operands (e.g., +AB)
- **Postfix (Reverse Polish):** Operator is after operands (e.g., AB+)

A **stack** is used to temporarily store operators and parentheses during the conversion.

IMPLEMENTATION:

- **PLATFORM:**
  - 64-bit Open source Linux or its derivatives.
  - Open Source C Programming tool like gcc/Eclipse Editor

- **TEST CONDITIONS:-**

1. Input infix expression and check the postfix or prefix output.
2. Input postfix or prefix expression and check its infix , prefix or postfix.

- **PSEUDO CODE:** ***//To be Written by Students***

Write pseudo code for push, pop, infix to postfix or infix to prefix(any one), prefix to infix , prefix to postfix, postfix to infix , postfix to prefix(any one which you implemented) operation.

- **TIME COMPLEXITY:** ***//To be Written by Students***

Write time complexity of above operations

- **CONCLUSION:**

Thus, implemented Stack Operations assignment using Array concepts. This System is able to apply different expression conversions.

- **FAQs** ***//To be Written by Students***

  - What is the advantage of prefix and postfix over infix expression?
  - Why is stack used in expression conversion?
  - Give stack full and stack empty conditions for stack.

- **PRACTICE ASSIGNMENTS**

1. Write a program to check whether a string is a palindrome or not using stack.
2. Write a program to convert decimal to binary using stack.